Reflection on NAND Universality:

NAND is called a universal gate because any basic logic gate can be constructed using only NAND gates. During the NAND2Tetris project, I learned that simple NAND operations can be combined to create NOT, AND, OR, XOR and other circuits. These basic circuits can then be combined to build adders and an ALU. This shows how complex computer hardware can ultimately be constructed from simple logic gates.

Nand 2 Tetris

Project 1

1)Not

Code: CHIP Not {

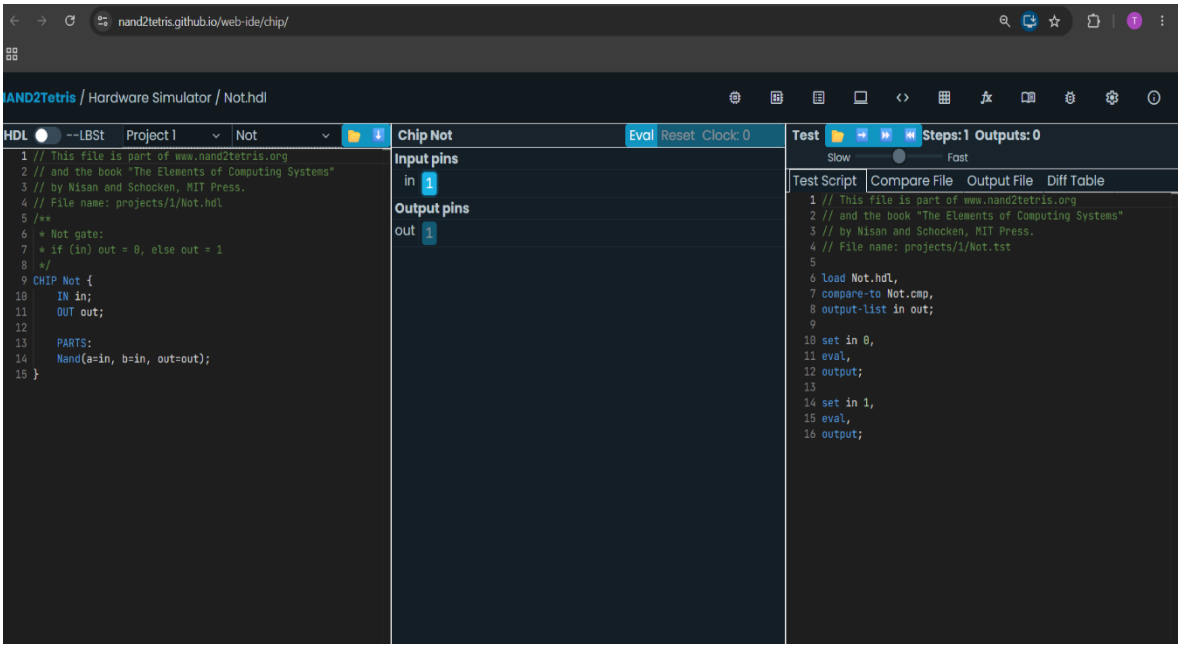
IN in;

OUT out;

PARTS:

Nand(a=in, b=in, out=out);

}



2)And

Code: CHIP And {

IN a, b;

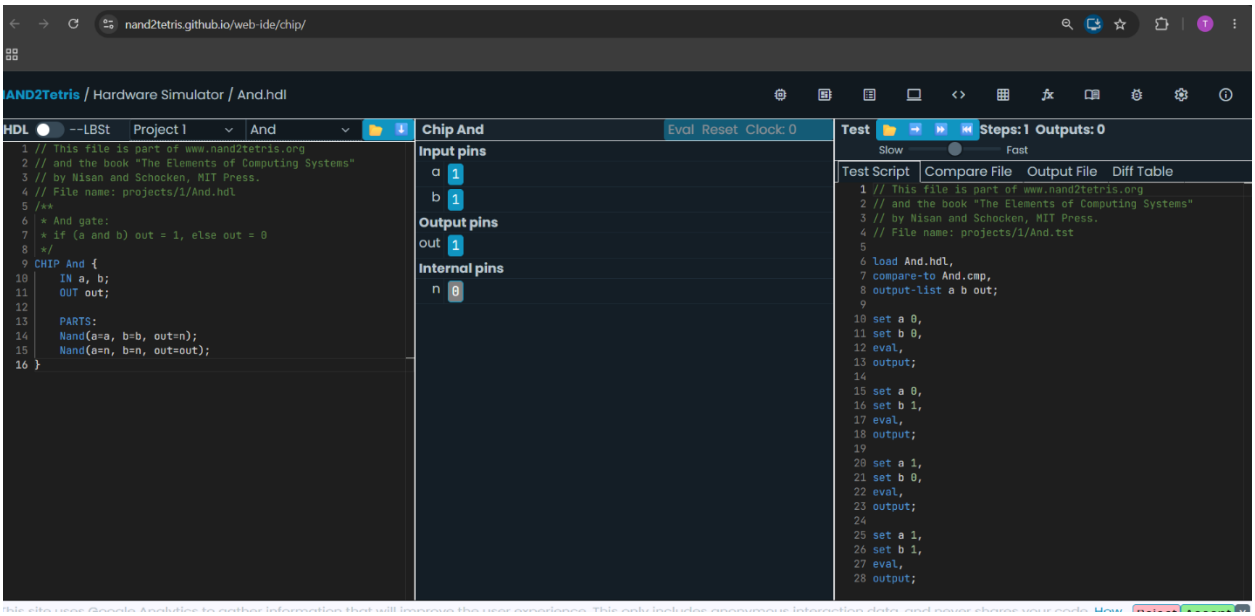
OUT out;

PARTS:

Nand(a=a, b=b, out=n);

Nand(a=n, b=n, out=out);

}



3)Or

Code: CHIP Or {

IN a, b;

OUT out;

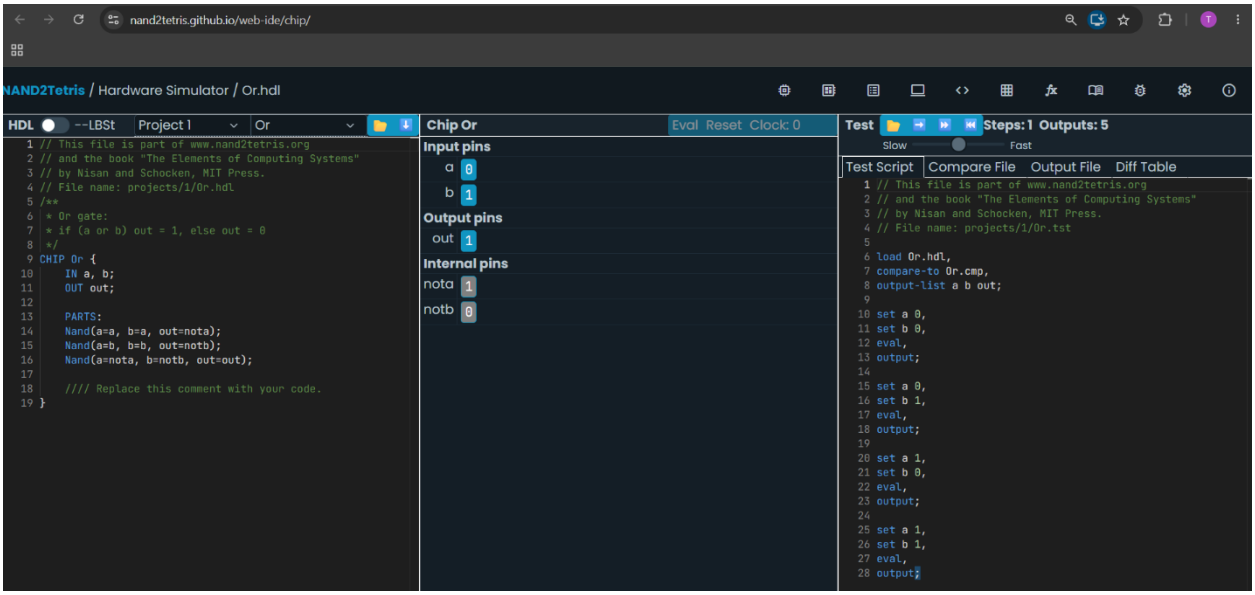
PARTS:

Nand(a=a, b=a, out=na);

Nand(a=b, b=b, out=nb);

Nand(a=na, b=nb, out=out);

}



4)Xor

Code CHIP Xor {

IN a, b;

OUT out;

PARTS:

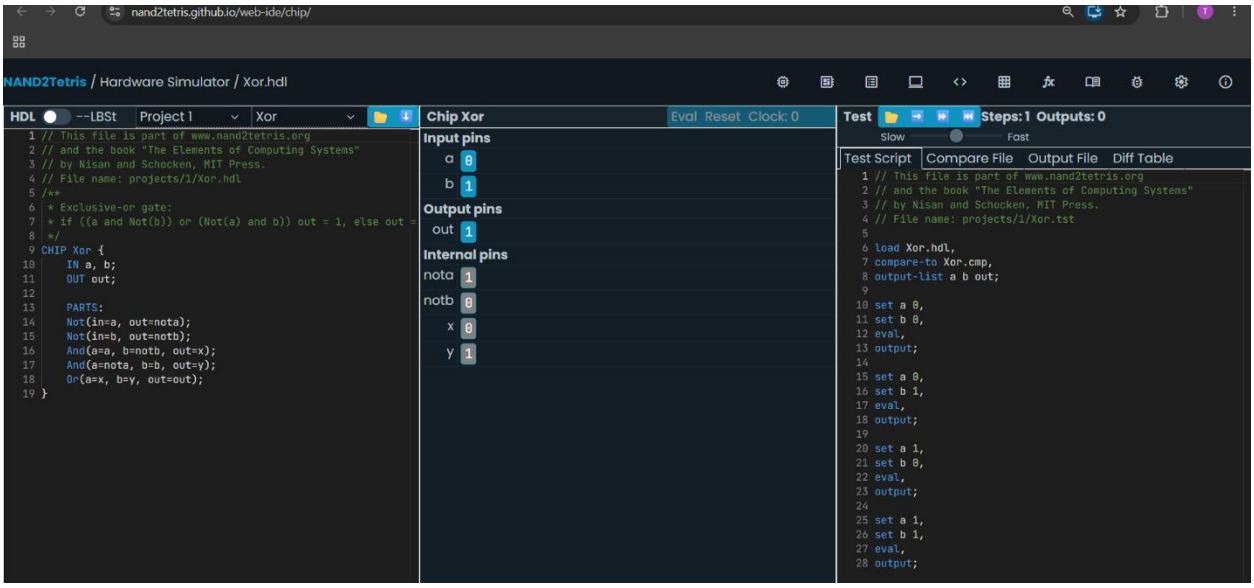
Nand(a=a, b=b, out=n1);

Nand(a=a, b=n1, out=n2);

Nand(a=b, b=n1, out=n3);

Nand(a=n2, b=n3, out=out);

}



5)Mux

CODE:CHIP Mux {

IN a, b, sel;

OUT out;

PARTS:

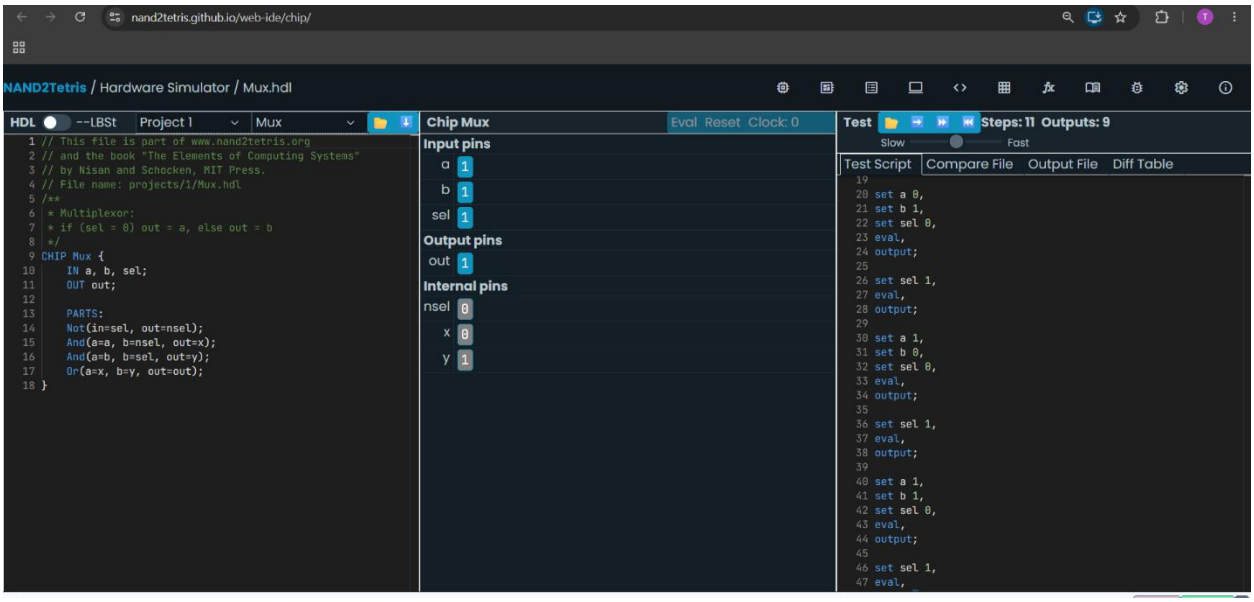
Not(in=sel, out=nsel);

And(a=a, b=nsel, out=x);

And(a=b, b=sel, out=y);

Or(a=x, b=y, out=out);

}



6)Dmux

CODE: CHIP DMux {

IN in, sel;

OUT a, b;

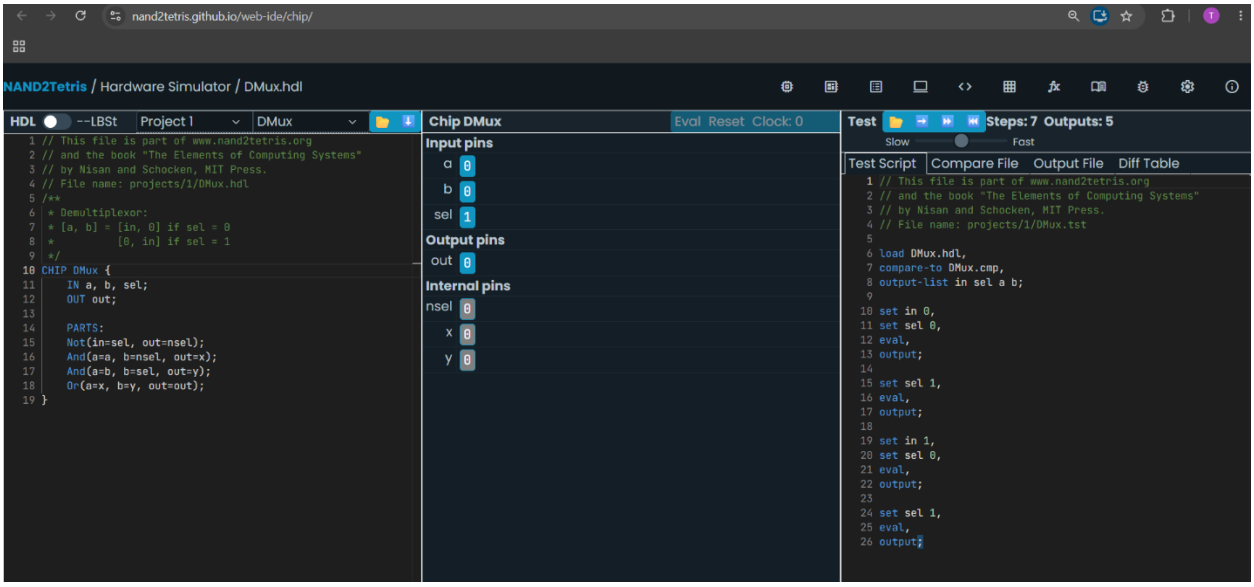
PARTS:

Not(in=sel, out=nsel);

And(a=in, b=nsel, out=a);

And(a=in, b=sel, out=b);

}



7)Not16

```
CHIP Not16 {

    IN in[16];

    OUT out[16];

PARTS:

Not(in=in[0], out=out[0]);

 Not(in=in[1], out=out[1]);

 Not(in=in[2], out=out[2]);

 Not(in=in[3], out=out[3]);

 Not(in=in[4], out=out[4]);

 Not(in=in[5], out=out[5]);

 Not(in=in[6], out=out[6]);

 Not(in=in[7], out=out[7]);

 Not(in=in[8], out=out[8]);

 Not(in=in[9], out=out[9]);

 Not(in=in[10], out=out[10]);

 Not(in=in[11], out=out[11]);

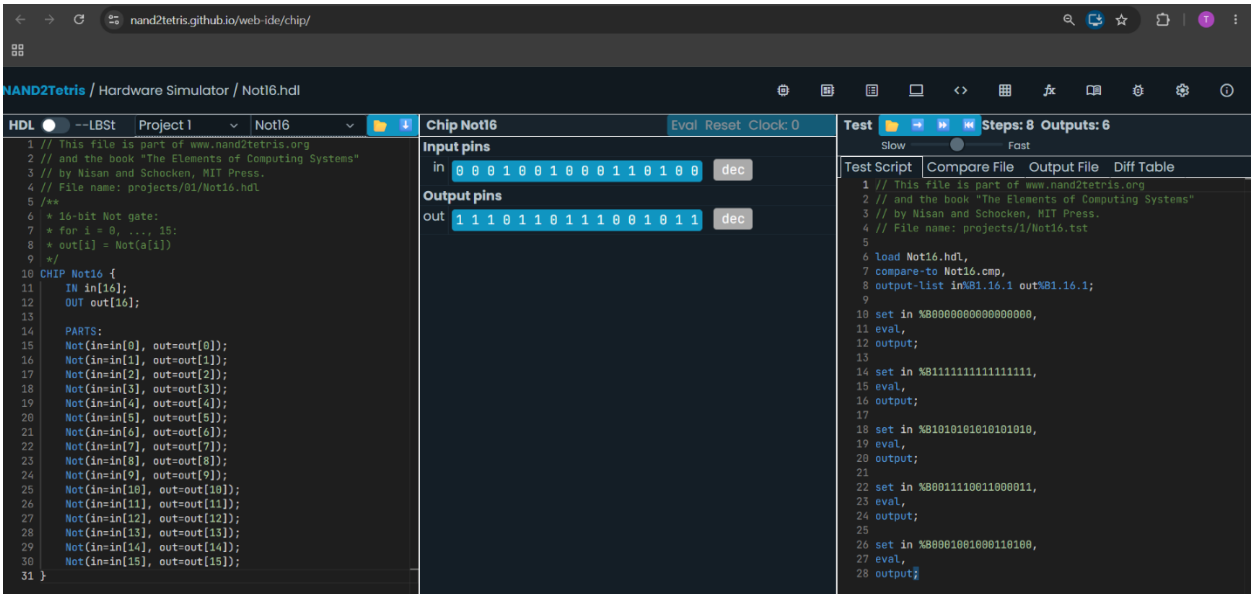
 Not(in=in[12], out=out[12]);

Not(in=in[13], out=out[13]);

 Not(in=in[14], out=out[14]);

Not(in=in[15], out=out[15]);

}
```


nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / Not16.hdl
HDL --LBSt Project 1 Not16
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/01/Not16.hdl
/**
* 16-bit Not gate:
* for i = 0, ..., 15:
* out[i] = Not(a[i])
*/
CHIP Not16 {
IN in[16];
OUT out[16];
PARTS:
Not(in=in[0], out=out[0]);
Not(in=in[1], out=out[1]);
Not(in=in[2], out=out[2]);
Not(in=in[3], out=out[3]);
Not(in=in[4], out=out[4]);
Not(in=in[5], out=out[5]);
Not(in=in[6], out=out[6]);
Not(in=in[7], out=out[7]);
Not(in=in[8], out=out[8]);
Not(in=in[9], out=out[9]);
Not(in=in[10], out=out[10]);
Not(in=in[11], out=out[11]);
Not(in=in[12], out=out[12]);
Not(in=in[13], out=out[13]);
Not(in=in[14], out=out[14]);
Not(in=in[15], out=out[15]);
}
Chip Not16
Eval Reset Clock: 0
Input pins
in 0 0 0 1 0 0 1 0 0 0 0 1 1 0 1 0 0
dec
Output pins
out 1 1 1 0 1 1 0 1 1 1 0 0 1 0 1 1
dec
Test
Steps: 8 Outputs: 6
Slow
Fast
Test Script
Compare File
Output File
Diff Table
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Not16.tst
load Not16.hdl,
compare-to Not16.cmp,
output-list in%B1.16.1 out%B1.16.1;
set in %B0000000000000000,
eval,
output;
set in %B1111111111111111,
eval,
output;
set in %B1010101010101010,
eval,
output;
set in %B0011110011000011,
eval,
output;
set in %B0001001000110100,
eval,
output;

8)And16

```
CHIP And16 {

    IN a[16], b[16];

    And(a=a[14], b=b[14], out=out[14]);

    And(a=a[15], b=b[15], out=out[15]);

    OUT out[16];


    PARTS:

    And(a=a[0], b=b[0], out=out[0]);

    And(a=a[1], b=b[1], out=out[1]);

    And(a=a[2], b=b[2], out=out[2]);

    And(a=a[3], b=b[3], out=out[3]);

    And(a=a[4], b=b[4], out=out[4]);

    And(a=a[5], b=b[5], out=out[5]);

    And(a=a[6], b=b[6], out=out[6]);

    And(a=a[7], b=b[7], out=out[7]);

    And(a=a[8], b=b[8], out=out[8]);

    And(a=a[9], b=b[9], out=out[9]);

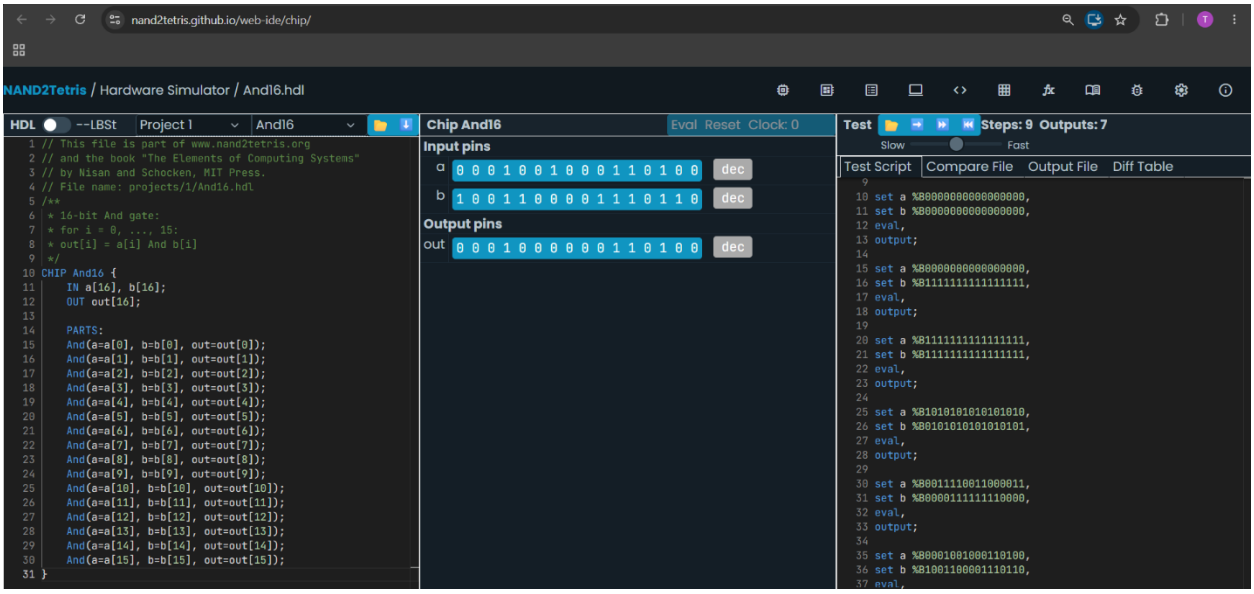
    And(a=a[10], b=b[10], out=out[10]);

    And(a=a[11], b=b[11], out=out[11]);

    And(a=a[12], b=b[12], out=out[12]);

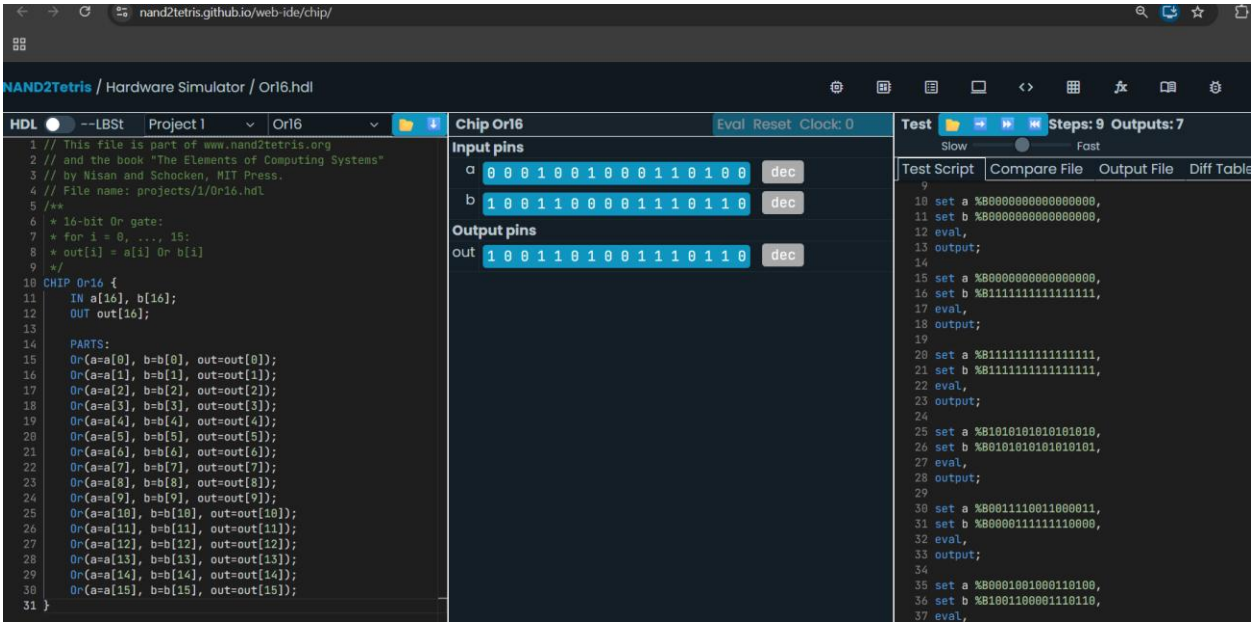
    And(a=a[13], b=b[13], out=out[13]);

}
```


nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / And16.hdl
HDL --LBSt Project 1 And16
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And16.hdl
/**
* 16-bit And gate:
* for i = 0, ..., 15:
* out[i] = a[i] And b[i]
*/
CHIP And16 {
IN a[16], b[16];
OUT out[16];
PARTS:
And(a=a[0], b=b[0], out=out[0]);
And(a=a[1], b=b[1], out=out[1]);
And(a=a[2], b=b[2], out=out[2]);
And(a=a[3], b=b[3], out=out[3]);
And(a=a[4], b=b[4], out=out[4]);
And(a=a[5], b=b[5], out=out[5]);
And(a=a[6], b=b[6], out=out[6]);
And(a=a[7], b=b[7], out=out[7]);
And(a=a[8], b=b[8], out=out[8]);
And(a=a[9], b=b[9], out=out[9]);
And(a=a[10], b=b[10], out=out[10]);
And(a=a[11], b=b[11], out=out[11]);
And(a=a[12], b=b[12], out=out[12]);
And(a=a[13], b=b[13], out=out[13]);
And(a=a[14], b=b[14], out=out[14]);
And(a=a[15], b=b[15], out=out[15]);
}
Chip And16
Eval Reset Clock: 0
Input pins
a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec
b 1 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 dec
Output pins
out 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 0 dec
Test
Steps: 9 Outputs: 7
Slow Fast
Test Script
Compare File
Output File
Diff Table
set a %B0000000000000000,
set b %B0000000000000000,
eval,
output;
set a %B0000000000000000,
set b %B1111111111111111,
eval,
output;
set a %B1111111111111111,
set b %B1111111111111111,
eval,
output;
set a %B1010101010101010,
set b %B0101010101010101,
eval,
output;
set a %B0011110011000011,
set b %B0000111111110000,
eval,
output;
set a %B0001001000110100,
set b %B1001100001110110,
eval,

9)Or16

```
CHIP Or16 {
    IN a[16], b[16];
    OUT out[16];
  PARTS:
    Or(a=a[0], b=b[0], out=out[0]);
    Or(a=a[1], b=b[1], out=out[1]);
    Or(a=a[2], b=b[2], out=out[2]);
    Or(a=a[3], b=b[3], out=out[3]);
    Or(a=a[4], b=b[4], out=out[4]);
    Or(a=a[5], b=b[5], out=out[5]);
    Or(a=a[6], b=b[6], out=out[6]);
    Or(a=a[7], b=b[7], out=out[7]);
    Or(a=a[8], b=b[8], out=out[8]);
    Or(a=a[9], b=b[9], out=out[9]);
    Or(a=a[10], b=b[10], out=out[10]);
    Or(a=a[11], b=b[11], out=out[11]);
    Or(a=a[12], b=b[12], out=out[12]);
    Or(a=a[13], b=b[13], out=out[13]);
    Or(a=a[14], b=b[14], out=out[14]);
    Or(a=a[15], b=b[15], out=out[15]);}
```


nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / Or16.hdl
HDL --LBSt Project 1 Or16
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or16.hdl
/**
* 16-bit Or gate:
* for i = 0, ..., 15:
* out[i] = a[i] Or b[i]
*/
CHIP Or16 {
IN a[16], b[16];
OUT out[16];
PARTS:
Or(a=a[0], b=b[0], out=out[0]);
Or(a=a[1], b=b[1], out=out[1]);
Or(a=a[2], b=b[2], out=out[2]);
Or(a=a[3], b=b[3], out=out[3]);
Or(a=a[4], b=b[4], out=out[4]);
Or(a=a[5], b=b[5], out=out[5]);
Or(a=a[6], b=b[6], out=out[6]);
Or(a=a[7], b=b[7], out=out[7]);
Or(a=a[8], b=b[8], out=out[8]);
Or(a=a[9], b=b[9], out=out[9]);
Or(a=a[10], b=b[10], out=out[10]);
Or(a=a[11], b=b[11], out=out[11]);
Or(a=a[12], b=b[12], out=out[12]);
Or(a=a[13], b=b[13], out=out[13]);
Or(a=a[14], b=b[14], out=out[14]);
Or(a=a[15], b=b[15], out=out[15]);
}
Chip Or16
Eval Reset Clock: 0
Input pins
a 0 0 0 1 0 0 1 0 0 0 0 1 1 0 1 0 0 dec
b 1 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 dec
Output pins
out 1 0 0 1 1 0 1 0 0 1 1 1 0 1 1 0 dec
Test Steps: 9 Outputs: 7
Slow Fast
Test Script Compare File Output File Diff Table
set a %B0000000000000000,
set b %B0000000000000000,
eval,
output;
set a %B0000000000000000,
set b %B1111111111111111,
eval,
output;
set a %B1111111111111111,
set b %B1111111111111111,
eval,
output;
set a %B1010101010101010,
set b %B0101010101010101,
eval,
output;
set a %B0011110011000011,
set b %B0000111111110000,
eval,
output;
set a %B0001001000110100,
set b %B1001100001110110,
eval,

10)Mux16

```
CHIP Mux16 {

    IN a[16], b[16], sel;

    OUT out[16];


    PARTS:

    Mux(a=a[0], b=b[0], sel=sel, out=out[0]);

    Mux(a=a[1], b=b[1], sel=sel, out=out[1]);

    Mux(a=a[2], b=b[2], sel=sel, out=out[2]);

    Mux(a=a[3], b=b[3], sel=sel, out=out[3]);

    Mux(a=a[4], b=b[4], sel=sel, out=out[4]);

    Mux(a=a[5], b=b[5], sel=sel, out=out[5]);

    Mux(a=a[6], b=b[6], sel=sel, out=out[6]);

    Mux(a=a[7], b=b[7], sel=sel, out=out[7]);

    Mux(a=a[8], b=b[8], sel=sel, out=out[8]);

    Mux(a=a[9], b=b[9], sel=sel, out=out[9]);

    Mux(a=a[10], b=b[10], sel=sel, out=out[10]);

    Mux(a=a[11], b=b[11], sel=sel, out=out[11]);

    Mux(a=a[12], b=b[12], sel=sel, out=out[12]);

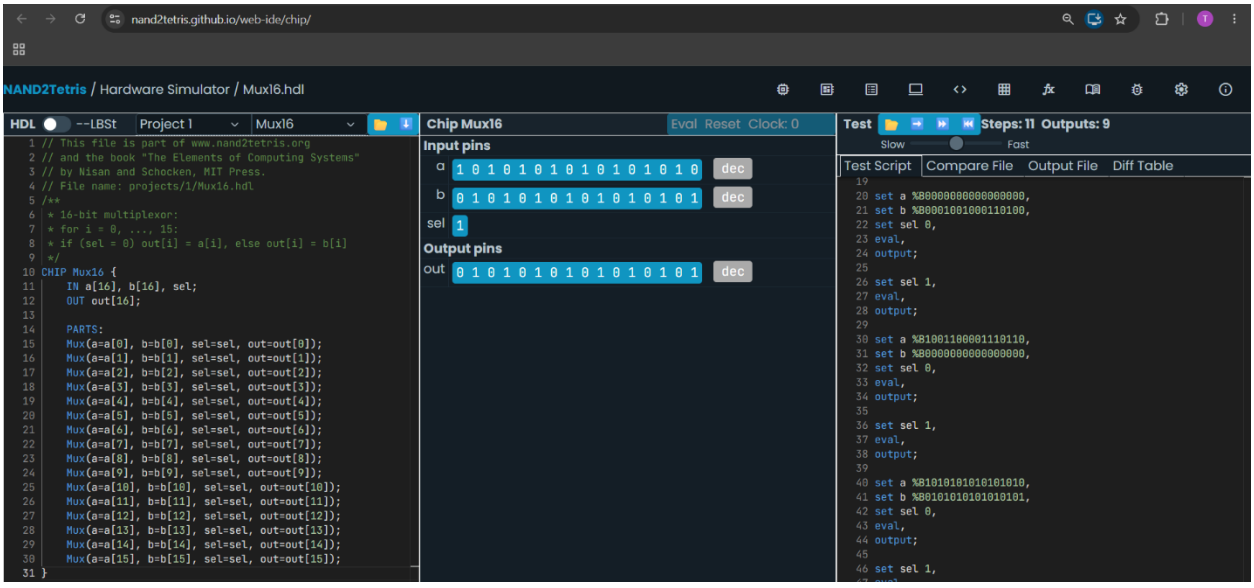
    Mux(a=a[13], b=b[13], sel=sel, out=out[13]);

     Mux(a=a[14], b=b[14], sel=sel, out=out[14]);

    Mux(a=a[15], b=b[15], sel=sel, out=out[15]);

}
```

```
    Mux(a=a[7], b=b[7], sel=sel, out=out[7]);
    Mux(a=a[8], b=b[8], sel=sel, out=out[8]);
    Mux(a=a[9], b=b[9], sel=sel, out=out[9]);
    Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
    Mux(a=a[11], b=b[11], sel=sel, out=out[11]);
    Mux(a=a[12], b=b[12], sel=sel, out=out[12]);
    Mux(a=a[13], b=b[13], sel=sel, out=out[13]);
    Mux(a=a[14], b=b[14], sel=sel, out=out[14]);
    Mux(a=a[15], b=b[15], sel=sel, out=out[15]);
}
```



11)Or8Way

```
CHIP Or8Way {

    IN in[8];

    OUT out;


    PARTS:

    Or(a=in[0], b=in[1], out=x1);

    Or(a=in[2], b=in[3], out=x2);

    Or(a=in[4], b=in[5], out=x3);

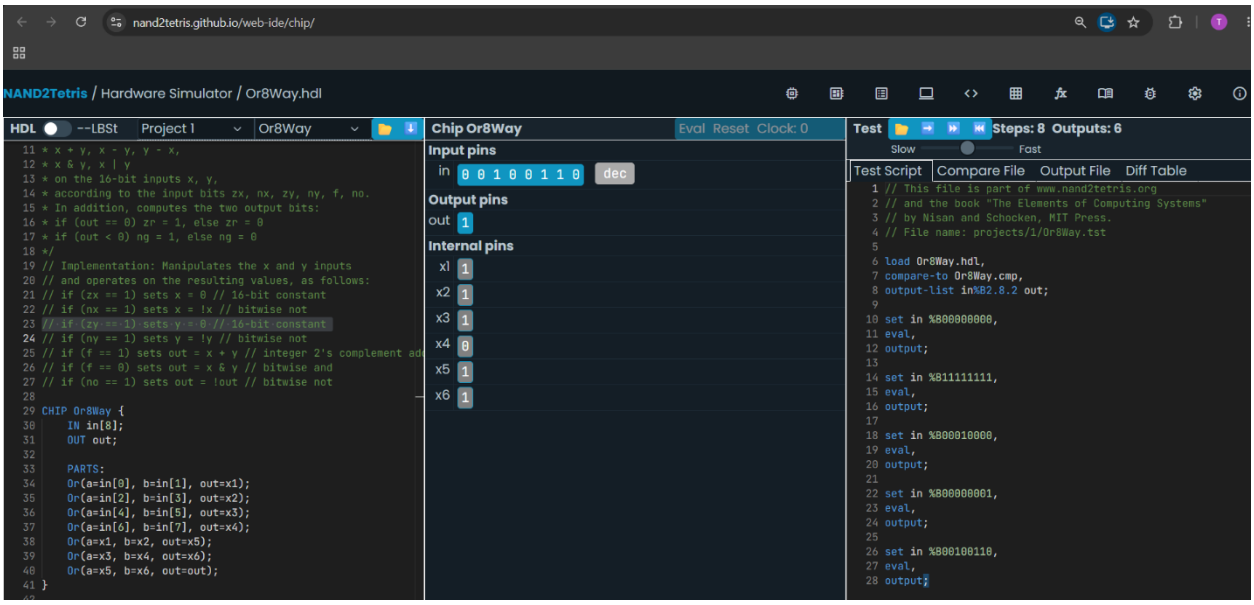
    Or(a=in[6], b=in[7], out=x4);

    Or(a=x1, b=x2, out=x5);

    Or(a=x3, b=x4, out=x6);

    Or(a=x5, b=x6, out=out);

}
```



12) Mux4Way16

CHIP Mux4Way16 {

IN a[16], b[16], c[16], d[16], sel[2];

OUT out[16];

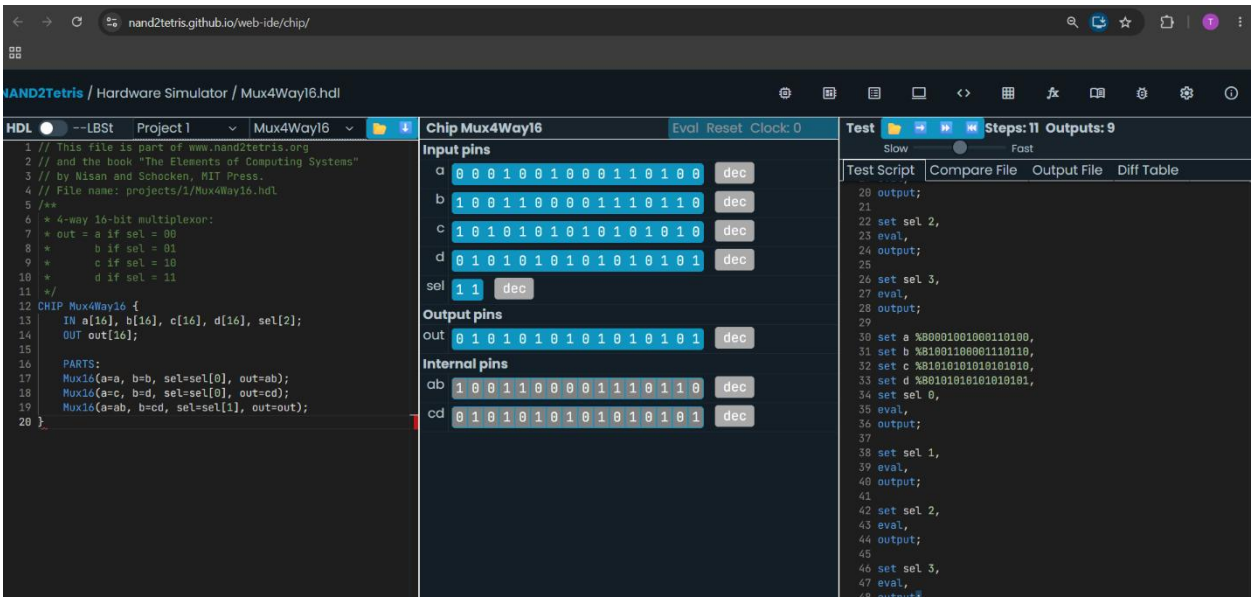
PARTS:

Mux16(a=a, b=b, sel=sel[0], out=ab);

Mux16(a=c, b=d, sel=sel[0], out=cd);

Mux16(a=ab, b=cd, sel=sel[1], out=out);

}



13) Mux8Way16

```
CHIP Mux8Way16 {

    IN a[16], b[16], c[16], d[16],

       e[16], f[16], g[16], h[16],

       sel[3];

    OUT out[16];

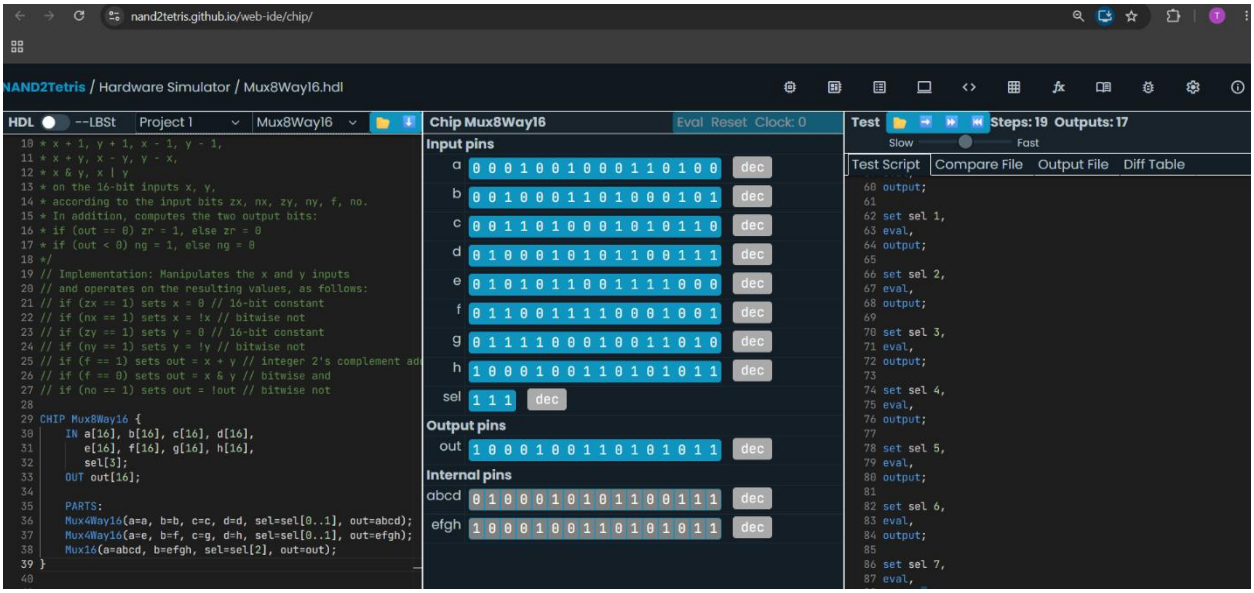

    PARTS:

    Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1], out=abcd);

    Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1], out=efgh);

    Mux16(a=abcd, b=efgh, sel=sel[2], out=out);

}
```



14)DMux4Way

CHIP DMux4Way {

IN in, sel[2];

OUT a, b, c, d;

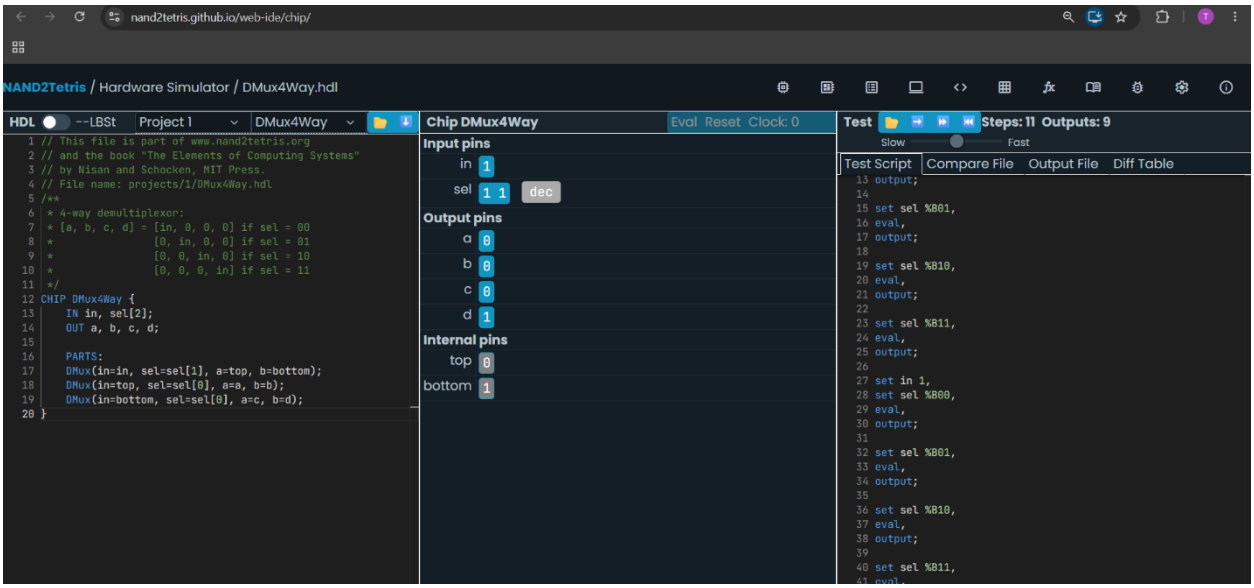
PARTS:

DMux(in=in, sel=sel[1], a=top, b=bottom);

DMux(in=top, sel=sel[0], a=a, b=b);

DMux(in=bottom, sel=sel[0], a=c, b=d);

}



15)Dmux8Way

CHIP DMux8Way {

IN in, sel[3];

OUT a, b, c, d, e, f, g, h;

PARTS:

DMux4Way(in=in, sel=sel[0..1], a=ab, b=cd, c=ef, d=gh);

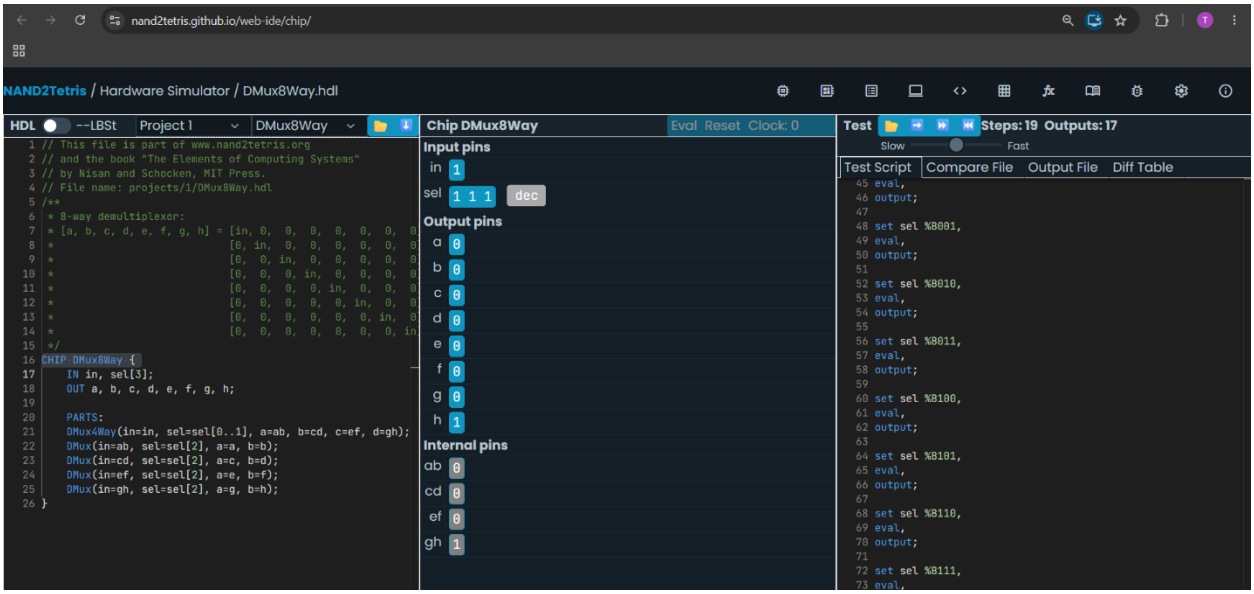
DMux(in=ab, sel=sel[2], a=a, b=b);

DMux(in=cd, sel=sel[2], a=c, b=d);

DMux(in=ef, sel=sel[2], a=e, b=f);

DMux(in=gh, sel=sel[2], a=g, b=h);

}

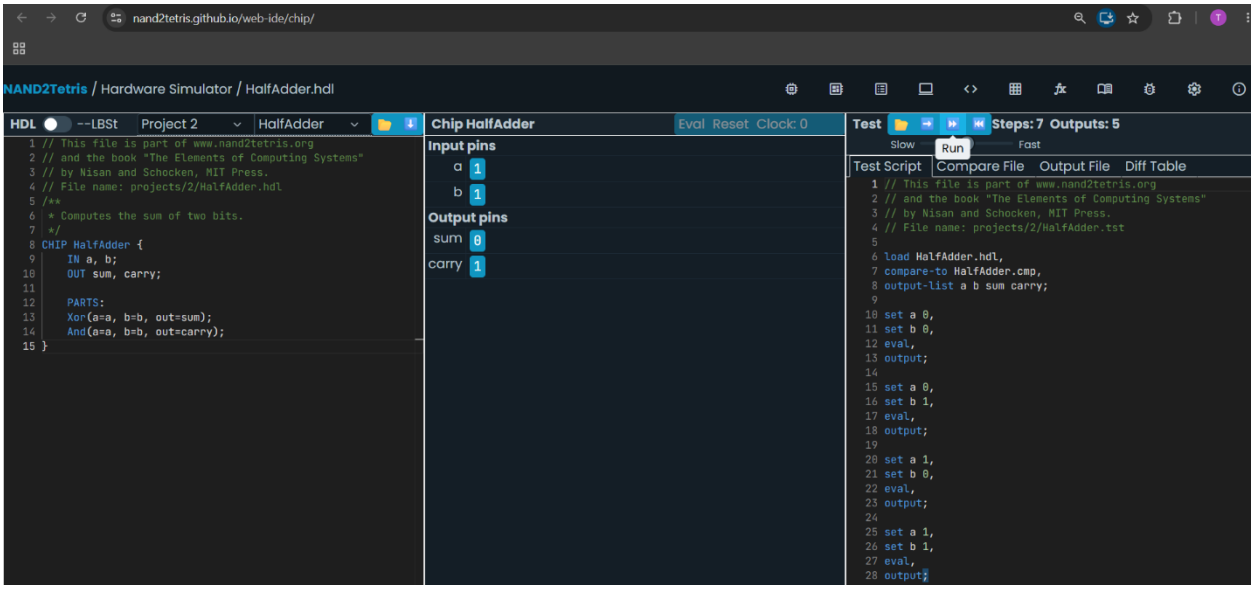


PROJECT2

1)HalfAdder

```
CHIP HalfAdder {
    IN a, b;
    OUT sum, carry;

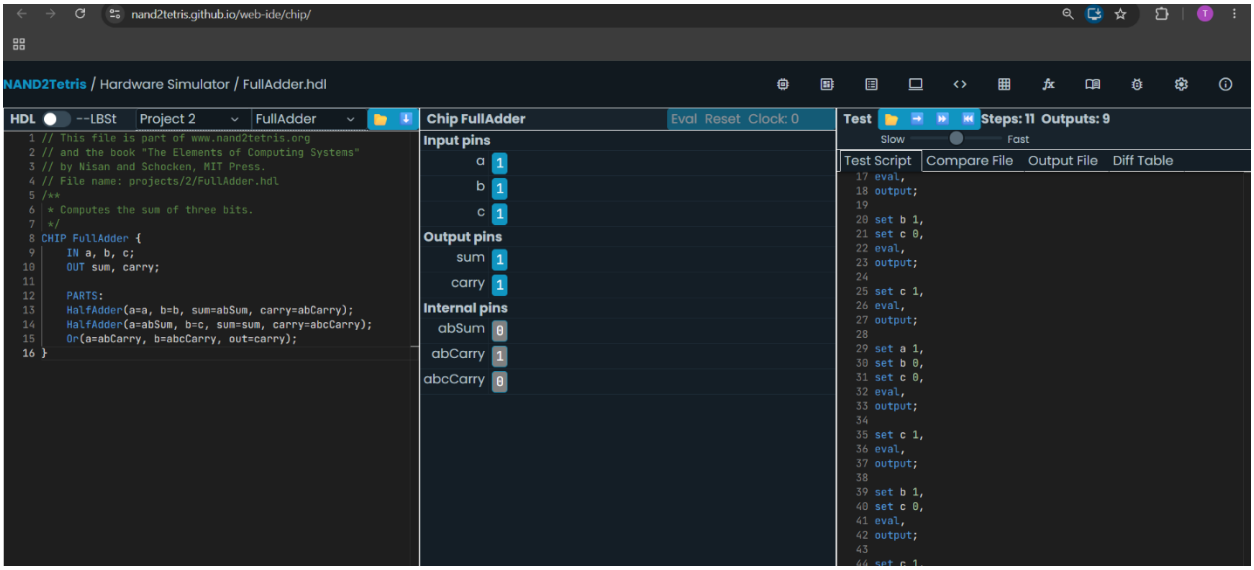
    PARTS:
    Xor(a=a, b=b, out=sum);
    And(a=a, b=b, out=carry);
}
```



2) FullAdder

```
CHIP FullAdder {
    IN a, b, c;
    OUT sum, carry;

    PARTS:
    HalfAdder(a=a, b=b, sum=abSum, carry=abCarry);
    HalfAdder(a=abSum, b=c, sum=sum, carry=abcCarry);
    Or(a=abCarry, b=abcCarry, out=carry);
}
```



3) Add16

```
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    FullAdder(a=a[0], b=b[0], c=false, sum=out[0], carry=c1);
    FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
    FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
    FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
    FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
    FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
    FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
```

```
        FullAdder(a=a[7], b=b[7], c=c7,
sum=out[7], carry=c8);

        FullAdder(a=a[8], b=b[8], c=c8,
sum=out[8], carry=c9);

        FullAdder(a=a[9], b=b[9], c=c9,
sum=out[9], carry=c10);

        FullAdder(a=a[10], b=b[10], c=c10,
sum=out[10], carry=c11);

        FullAdder(a=a[11], b=b[11], c=c11,
sum=out[11], carry=c12);

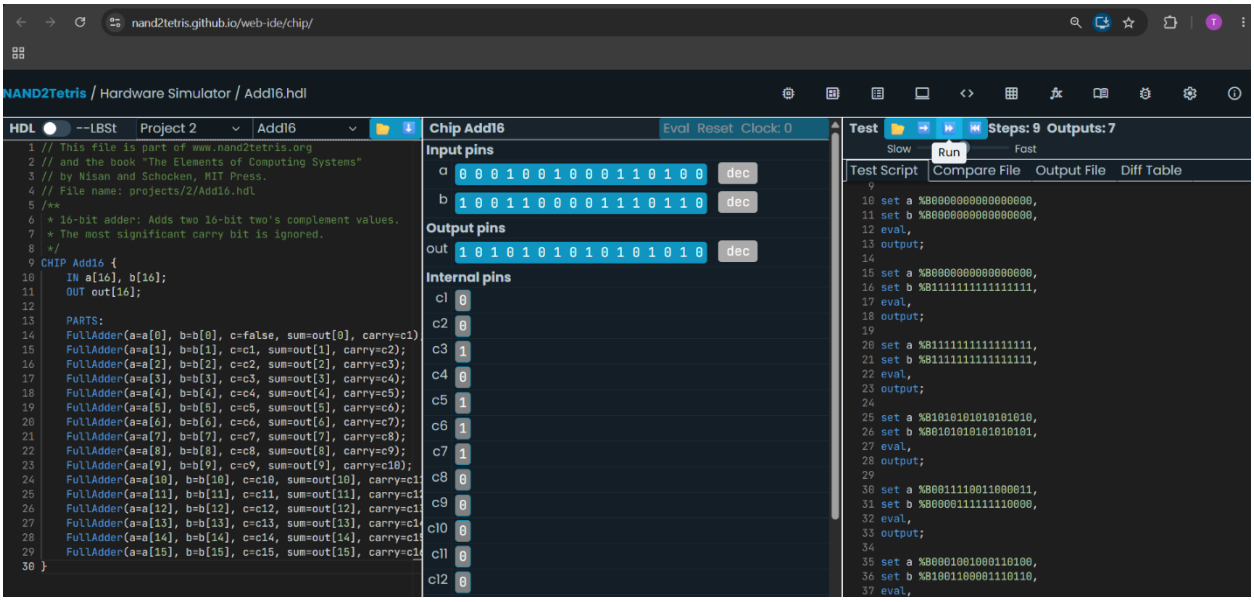
        FullAdder(a=a[12], b=b[12], c=c12,
sum=out[12], carry=c13);

        FullAdder(a=a[13], b=b[13], c=c13,
sum=out[13], carry=c14);

        FullAdder(a=a[14], b=b[14], c=c14,
sum=out[14], carry=c15);

        FullAdder(a=a[15], b=b[15], c=c15,
sum=out[15], carry=c16);

    }
```


nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / Add16.hdl
HDL --LBSt Project 2 Add16
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/Add16.hdl
/**
* 16-bit adder: Adds two 16-bit two's complement values.
* The most significant carry bit is ignored.
*/
CHIP Add16 {
IN a[16], b[16];
OUT out[16];
PARTS:
FullAdder(a=a[0], b=b[0], c=false, sum=out[0], carry=c1);
FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
FullAdder(a=a[7], b=b[7], c=c7, sum=out[7], carry=c8);
FullAdder(a=a[8], b=b[8], c=c8, sum=out[8], carry=c9);
FullAdder(a=a[9], b=b[9], c=c9, sum=out[9], carry=c10);
FullAdder(a=a[10], b=b[10], c=c10, sum=out[10], carry=c11
FullAdder(a=a[11], b=b[11], c=c11, sum=out[11], carry=c12
FullAdder(a=a[12], b=b[12], c=c12, sum=out[12], carry=c13
FullAdder(a=a[13], b=b[13], c=c13, sum=out[13], carry=c14
FullAdder(a=a[14], b=b[14], c=c14, sum=out[14], carry=c15
FullAdder(a=a[15], b=b[15], c=c15, sum=out[15], carry=c16
}
Chip Add16
Eval Reset Clock: 0
Input pins
a 0 0 0 1 0 0 1 0 0 0 0 1 1 0 1 0 0 dec
b 1 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 dec
Output pins
out 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 dec
Internal pins
c1 0
c2 0
c3 1
c4 0
c5 1
c6 1
c7 1
c8 0
c9 0
c10 0
c11 0
c12 0
Test
Steps: 9 Outputs: 7
Slow
Run
Fast
Test Script
Compare File
Output File
Diff Table
set a %B0000000000000000,
set b %B0000000000000000,
eval,
output;
set a %B0000000000000000,
set b %B1111111111111111,
eval,
output;
set a %B1111111111111111,
set b %B1111111111111111,
eval,
output;
set a %B1010101010101010,
set b %B0101010101010101,
eval,
output;
set a %B0011110011000011,
set b %B0000111111110000,
eval,
output;
set a %B0001001000110100,
set b %B1001100001110110,
eval,

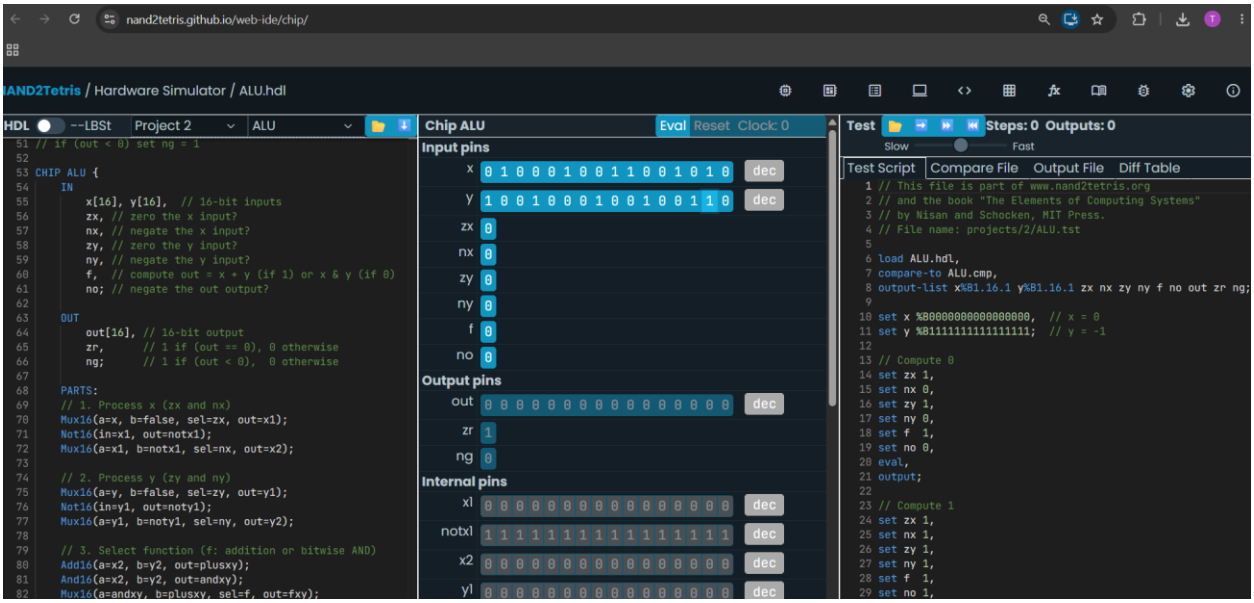
4) ALU

```
CHIP ALU {
    IN
        x[16], y[16],  // 16-bit inputs
    OUT
        out[16],
PARTS:
    Mux16(a=x, b=false, sel=zx, out=x1);
    Not16(in=x1, out=notx1);
    Mux16(a=x1, b=notx1, sel=nx, out=x2);
 Mux16(a=y, b=false, sel=zy, out=y1);
    Not16(in=y1, out=noty1);
    Mux16(a=y1, b=noty1, sel=ny, out=y2);
Add16(a=x2, b=y2, out=plusxy);
    And16(a=x2, b=y2, out=andxy);
    Mux16(a=andxy, b=plusxy, sel=f, out=fxy);
Not16(in=fxy, out=notfxy);
    Mux16(a=fxy, b=notfxy, sel=no, out=out, out[0..7]=outLow, out[8..15]=outHigh, out[15]=signBit);
    Or8Way(in=outLow, out=orLow);
    Or8Way(in=outHigh, out=orHigh);
    Or(a=orLow, b=orHigh, out=notZr);
    Not(in=notZr, out=zr);



    And(a=signBit, b=true, out=ng);
}
```

zx	nx	zy	ny	f	no	Function
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	0	1	0	x
1	0	0	0	1	0	y
0	0	1	1	1	1	!x
1	1	0	0	1	1	!y
0	0	1	1	1	0	-x
1	1	0	0	1	0	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x|y


nand2tetris.github.io/web-ide/chip/
AND2Tetris / Hardware Simulator / ALU.hdl
HDL --LBSt Project 2 ALU
// if (out < 0) set ng = 1
CHIP ALU {
IN
x[16], y[16], // 16-bit inputs
zx, // zero the x input?
nx, // negate the x input?
zy, // zero the y input?
ny, // negate the y input?
f, // compute out = x + y (if 1) or x & y (if 0)
no; // negate the out output?
OUT
out[16], // 16-bit output
zr, // 1 if (out == 0), 0 otherwise
ng; // 1 if (out < 0), 0 otherwise
PARTS:
// 1. Process x (zx and nx)
Mux16(a=x, b=false, sel=zx, out=x1);
Not16(in=x1, out=notx1);
Mux16(a=x1, b=notx1, sel=nx, out=x2);
// 2. Process y (zy and ny)
Mux16(a=y, b=false, sel=zy, out=y1);
Not16(in=y1, out=noty1);
Mux16(a=y1, b=noty1, sel=ny, out=y2);
// 3. Select function (f: addition or bitwise AND)
Add16(a=x2, b=y2, out=plusxy);
And16(a=x2, b=y2, out=andxy);
Mux16(a=andxy, b=plusxy, sel=f, out=fxy);
Chip ALU
Eval Reset Clock: 0
Input pins
x 0 1 0 0 0 1 0 0 1 1 0 0 1 0 1 0 dec
y 1 0 0 1 0 0 0 1 0 0 1 0 0 1 1 0 dec
zx 0
nx 0
zy 0
ny 0
f 0
no 0
Output pins
out 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec
zr 1
ng 0
Internal pins
x1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec
notx1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 dec
x2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec
y1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 dec
Test Steps: 0 Outputs: 0
Slow Fast
Test Script Compare File Output File Diff Table
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/ALU.tst
load ALU.hdl,
compare-to ALU.cmp,
output-list x%B1.16.1 y%B1.16.1 zx nx zy ny f no out zr ng;
set x %B0000000000000000, // x = 0
set y %B1111111111111111; // y = -1
// Compute 0
set zx 1,
set nx 0,
set zy 1,
set ny 0,
set f 1,
set no 0,
eval,
output;
// Compute 1
set zx 1,
set nx 1,
set zy 1,
set ny 1,
set f 1,
set no 1,